

DOCUMENTAÇÃO DE PROJETO

TravelHub - Sistema Inteligente de Gestão de Eventos e Viagens em Grupo

1. Visão Geral do Projeto

Objetivo

Desenvolver uma plataforma web para gerenciamento de viagens, excursões, eventos e encontros em grupo, permitindo o controle de participantes, despesas, pagamentos, cronogramas, votações e estatísticas.

O sistema será construído inicialmente com foco no desenvolvimento Backend utilizando Java e os conceitos de Programação Orientada a Objetos (POO), permitindo posteriormente a integração com um Frontend desenvolvido em React.

2. Problema que o Sistema Resolve

Atualmente, grupos de amigos, familiares ou equipes costumam organizar viagens utilizando múltiplas ferramentas:

- WhatsApp para comunicação;
- Planilhas para controle financeiro;
- Aplicativos de notas para cronogramas;
- Enquetes para decisões.

Isso gera descentralização das informações, dificultando o gerenciamento do evento.

O TravelHub centraliza todas essas funcionalidades em uma única plataforma.

3. Escopo do Sistema

O sistema deverá permitir:

Gestão de Usuários

- Cadastro de usuários;
- Login e autenticação;
- Atualização de perfil;
- Controle de permissões.

Gestão de Eventos

- Criação de eventos;
- Edição de eventos;
- Cancelamento de eventos;
- Consulta de eventos.

Gestão de Participantes

- Convite de participantes;
- Entrada em eventos;
- Remoção de participantes;
- Controle de presença.

Gestão Financeira

- Cadastro de despesas;
- Divisão automática de custos;
- Controle de pagamentos;
- Histórico financeiro.

Sistema de Votação

- Criação de votações;
- Registro de votos;
- Apuração automática.

Cronograma

- Cadastro de atividades;
- Organização por data e horário;
- Visualização em timeline.

Dashboard

- Indicadores financeiros;
 - Quantidade de participantes;
 - Eventos ativos;
 - Estatísticas gerais.
-

4. Requisitos Funcionais

RF001 – Cadastro de Usuários

O sistema deverá permitir o cadastro de novos usuários.

RF002 – Autenticação

O sistema deverá permitir login utilizando email e senha.

RF003 – Criação de Eventos

Usuários autenticados poderão criar eventos.

RF004 – Gerenciamento de Participantes

O organizador poderá adicionar ou remover participantes.

RF005 – Cadastro de Despesas

Participantes autorizados poderão cadastrar despesas relacionadas ao evento.

RF006 – Divisão de Custos

O sistema deverá calcular automaticamente o valor individual devido por cada participante.

RF007 – Registro de Pagamentos

O sistema deverá registrar pagamentos efetuados.

RF008 – Sistema de Votação

O sistema deverá permitir votações entre participantes.

RF009 – Gestão de Cronograma

O sistema deverá permitir o cadastro de atividades do evento.

RF010 – Dashboard

O sistema deverá exibir métricas e estatísticas do evento.

5. Requisitos Não Funcionais

RNF001

O sistema deverá possuir autenticação baseada em JWT.

RNF002

O sistema deverá utilizar banco de dados PostgreSQL.

RNF003

As APIs deverão seguir o padrão REST.

RNF004

O sistema deverá utilizar arquitetura em camadas.

RNF005

O código deverá seguir princípios SOLID.

RNF006

O sistema deverá possuir documentação da API utilizando Swagger/OpenAPI.

6. Tecnologias Utilizadas

Backend

- Java 21
- Spring Boot
- Spring Data JPA
- Hibernate
- Spring Security
- JWT
- Maven

Banco de Dados

- PostgreSQL

Frontend

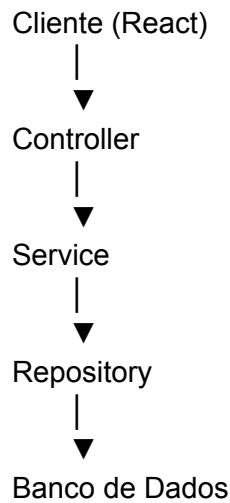
- React
- React Router
- Axios
- Bootstrap

Ferramentas

- Git
 - GitHub
 - Postman
 - Docker
-

7. Arquitetura do Projeto

O projeto seguirá uma arquitetura em camadas.



Controller Layer

Responsável por receber requisições HTTP.

Exemplo:

```
@RestController
@RequestMapping("/usuarios")
public class UsuarioController
```

Service Layer

Responsável pelas regras de negócio.

Exemplo:

```
@Service
public class UsuarioService
```

Repository Layer

Responsável pelo acesso aos dados.

Exemplo:

@Repository
public interface UsuarioRepository

8. Estrutura de Pastas

src/main/java

com.travelhub

- |— controller
- |— service
- |— repository
- |— model
- |— dto
- |— mapper
- |— exception
- |— security
- |— config
- |— util

9. Modelagem Inicial

Entidade Usuario

```
public class Usuario {  
  
    private Long id;  
  
    private String nome;  
  
    private String email;  
  
    private String senha;  
  
    private LocalDateTime dataCadastro;  
  
}
```

Entidade Evento

```
public class Evento {
```

```
private Long id;

private String nome;

private String descricao;

private String destino;

private LocalDate dataInicio;

private LocalDate dataFim;

}
```

Entidade Participante

```
public class Participante {

    private Long id;

    private Usuario usuario;

    private Evento evento;

    private StatusPagamento statusPagamento;

}
```

Entidade Despesa

```
public class Despesa {

    private Long id;

    private String descricao;

    private BigDecimal valor;

    private Usuario responsavel;

    private Evento evento;

}
```

Entidade Votacao

```
public class Votacao {  
  
    private Long id;  
  
    private String titulo;  
  
    private Evento evento;  
  
}
```

Entidade OpcaoVoto

```
public class OpcaoVoto {  
  
    private Long id;  
  
    private String descricao;  
  
}
```

10. Conceitos de POO Aplicados

Encapsulamento

Todos os atributos deverão ser privados.

```
private String nome;
```

Herança

```
Usuario  
|  
├── Administrador  
└── Participante
```

Polimorfismo

Pagamento

Implementações:

PixPagamento

CartaoPagamento

DinheiroPagamento

Abstração

public interface CalculadoraDespesa

Implementações:

CalculadoraIgualitaria

CalculadoraPorPeso

11. Design Patterns

Strategy

Responsável pela divisão de despesas.

DivisaoStrategy

Implementações:

DivisaoIgualitaria

DivisaoProporcional

Factory

Criação de tipos de eventos.

EventoFactory

Builder

Construção de eventos complexos.

Singleton

Configurações globais do sistema.

12. Endpoints da API

Usuários

Criar Usuário

POST /usuarios

Buscar Usuários

GET /usuarios

Buscar Usuário por ID

GET /usuarios/{id}

Atualizar Usuário

PUT /usuarios/{id}

Excluir Usuário

DELETE /usuarios/{id}

Eventos

Criar Evento

POST /eventos

Listar Eventos

GET /eventos

Despesas

Criar Despesa

POST /despesas

Listar Despesas

GET /despesas

13. Segurança

O sistema utilizará autenticação baseada em JWT.

Fluxo:

Usuário faz login



Spring Security



JWT Token



Retorno para Cliente



Requisições autenticadas

Endpoint de Login

POST /auth/login

Exemplo de resposta:

```
{  
  "token": "eyJhbGciOi..."  
}
```

14. Roadmap de Desenvolvimento

Sprint 1 – Fundação

Objetivos:

- Configurar Spring Boot;
- Configurar PostgreSQL;
- Criar entidades;
- Configurar JPA.

Duração estimada:

7 dias.

Sprint 2 – CRUD Completo

Objetivos:

- Usuários;
- Eventos;
- Participantes.

Duração estimada:

7 dias.

Sprint 3 – Regras de Negócio

Objetivos:

- Divisão de despesas;
- Controle financeiro;
- Votações.

Duração estimada:

10 dias.

Sprint 4 – Segurança

Objetivos:

- Spring Security;
- JWT;
- Controle de acesso.

Duração estimada:

5 dias.

Sprint 5 – Frontend

Objetivos:

- Dashboard;
- Eventos;
- Financeiro;
- Votações.

Duração estimada:

10 dias.

Sprint 6 – Deploy

Objetivos:

- Docker;
- Banco em produção;
- Deploy Backend;
- Deploy Frontend.

Duração estimada:

5 dias.

15. Evoluções Futuras

Módulo de Inteligência Artificial

Sugestão automática de:

- Destinos;
 - Roteiros;
 - Cronogramas.
-

Sistema de Notificações

- Email;
 - WhatsApp;
 - Push Notification.
-

Dashboard Analítico

- Gastos por categoria;
 - Participação dos usuários;
 - Histórico de eventos.
-

Microserviços

Separação dos módulos:

Auth Service

Evento Service

Financeiro Service

Notification Service

Mensageria

Integração com:

- RabbitMQ
- Apache Kafka